Machine Coding: Error Boundaries

- **Problem:** Build a reusable Error Boundary system so one broken widget never takes down the whole page.
- **Tech stack:** React 19 (Vite, JavaScript) *(default)*
- **Time limit:** 60 minutes *(default)*

PART A: Thought process

Steps that work for any machine coding task:

1. **Understand the requirements:** functional, non-functional, clarifying questions, assumptions.
2. **List the interactions:** every action and its expected result. This becomes the checklist.
3. **Design the architecture:** components and who owns what.
4. **Design the state:** the smallest state; derive the rest.
5. **Define data and contracts:** props, payloads, mock APIs.
6. **Trace one flow end to end** before coding.
7. **Find the hard parts** and decide how to solve them.
8. **Build in layers:** simplest working version first, then add one working layer at a time.
9. **Cover states and edge cases:** initial, loading, success, empty, error.
10. **Wrap up:** trade-offs and what you would add with more time.

Time budget (60 min)

Phase	Minutes
Requirements + questions	5
Use cases + architecture + state	8
Core `ErrorBoundary` class + fallback	12
Demo widgets (render / async / event errors)	15
Reset (retry button + `resetKeys`) + logging	10
Edge cases, polish, accessibility	5
Wrap-up and trade-offs	5

Think out loud. Say *why* before you type: "Boundaries only catch render errors, so for the fetch I'll re-throw from state." The interviewer grades the reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A reusable `<ErrorBoundary>` that catches errors thrown while rendering its subtree and shows a fallback UI instead of a blank page.
- A fallback that shows the error message and a **Try again** button that resets the boundary.
- A custom fallback can be passed: a React element or a render function `({ error, reset }) => node`.
- **Granular isolation:** each widget has its own boundary; a root boundary is the last line of defence.
- **Errors are logged** (message, stack, component stack, boundary name) to a logging service.
- **Auto reset** when given `resetKeys` change (e.g. the user picks another item).
- A way to send **async and event-handler errors** to the nearest boundary (`useErrorBoundary`).

Non-functional requirements

- **Error handling:** a crash in one widget leaves the rest of the page usable.
- **Reusability:** one generic boundary; behaviour comes from props (`fallback` , `onError` , `onReset` , `resetKeys` , `name`).
- **Extensibility:** logger is a module that can be swapped for Sentry without touching components.
- **Accessibility:** fallback uses `role="alert"` , retry is a real `<button>` , focus styles visible.
- **Performance:** zero cost when there is no error (boundary just returns `children`).
- **Security:** stack traces shown only in development.

Clarifying questions

1. Should only render errors be caught, or also async (fetch) and event-handler errors?
2. How granular: one boundary for the whole app, per route, or per widget?
3. What should recovery look like: retry button, auto reset when inputs change, or full reload?
4. Where do errors get reported: console, a backend endpoint, or a tool like Sentry?
5. Should the user see technical details (stack trace) or only a friendly message?
6. Can I write my own boundary, or is a library like `react-error-boundary` allowed?

Assumptions

- Write the boundary ourselves (no library) to show the internals.
- Catch render errors natively; route async and event errors through a `useErrorBoundary` hook.
- Per-widget boundaries plus one root boundary.
- Logging is a mock module (console + in-memory array).
- Stack trace inside a `<details>` only in dev (`import.meta.env.DEV`).

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads, no errors	All widgets render normally

Action / input	Expected behavior / output
Counter reaches 3 (throws during render)	Only the counter card shows the fallback; other cards keep working
Click **Try again** in a fallback	Boundary resets, child remounts with fresh state (counter back to 0)
Fetch rejects (Fail request checked)	Error is re-thrown from state, user card shows fallback
Switch user / uncheck Fail request while failing	`resetKeys` changed, boundary auto resets, card loads again
Event handler throws, handled inline	Inline red message, no fallback (boundaries don't catch handlers)
Event handler throws, escalated with `showBoundary`	Card shows fallback
Crash in `App` itself (outside widget boundaries)	Root boundary shows a full page "The app crashed" with retry
Any caught error	`logError` is called with message, stack, component stack, boundary name
Fallback itself throws	Error goes up to the next parent boundary

3. Design the architecture

Module	Responsibility
`ErrorBoundary` (class)	Catch render errors, switch to fallback, log, reset
`ErrorFallback`	Default accessible fallback UI with retry
`useErrorBoundary`	Hook that pushes async/event errors into the nearest boundary
`errorLogger`	Report errors (mock of Sentry / beacon)
`resetKeys` util	Pure compare of reset keys
`api`	Mock `fetchUser` that can fail
`BuggyCounter`, `UserCard`, `SaveButton`	Demo widgets: render, async and event errors
`App`	Layout, one boundary per widget, controls outside boundaries

```
main.jsx
└── ErrorBoundary name="root"        (full-page fallback)
    └── App
        ├── header  [Crash whole app]
        ├── card: ErrorBoundary "counter"
        │         └── BuggyCounter
        ├── card: controls (user select, fail checkbox)
        │         ErrorBoundary "user-card" resetKeys=[userId, failFetch]
        │         └── UserCard key=userId   (uses useErrorBoundary)
        └── card: ErrorBoundary "save"
```

```
                └── SaveButton            (uses useErrorBoundary)
ErrorBoundary ──> errorLogger.logError()
```

Start simple: one boundary class + one fallback. Split out the hook, logger and reset util only once their jobs are clear.

4. Design the state / data model

State or field	Owner	Purpose
`hasError`	ErrorBoundary	Show children or fallback
`error`	ErrorBoundary	Passed to the fallback and logger
`error`	useErrorBoundary (per child)	Holds an async error so the next render throws it
`userId` , `failFetch`	App	Drive the user card and its `resetKeys`
`crashApp`	App	Demo a crash caught by the root boundary
`count`	BuggyCounter	Demo render error
`user`	UserCard	Loaded user or `null` (loading)
`message`	SaveButton	Inline error text for handled event errors

```
const initialState = { hasError: false, error: null };

export default class ErrorBoundary extends Component {
  state = initialState;
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }
}
```

Senior note

- **Why a separate `hasError` flag?** `throw null` or `throw undefined` is legal JS. With only `error` , `error === null` would hide the crash.
- **Derived, not stored:** "is loading" is `user === null` ; no extra `loading` flag. Previous reset keys come from `prevProps` , not from state.
- **Ownership:** the boundary owns only error state. Controls (`userId` , `failFetch`) live *above* the boundary in `App` so they survive the crash and can be used to recover.

5. Design the data structures and contracts

ErrorBoundary props contract

Prop	Type	Meaning
`children`	node	Protected subtree

Prop	Type	Meaning
`fallback`	node or `({ error, reset }) => node`	Custom fallback (default `ErrorFallback`)
`onError`	`(error, info) => void`	Extra hook after logging
`onReset`	`() => void`	Clear parent state that caused the error
`resetKeys`	`unknown[]`	Auto reset when any key changes
`name`	string	Boundary label in logs

Mock API

```
// fetchUser(id, shouldFail) -> Promise
// resolves after 600ms:
{ id: 1, name: 'Ada Lovelace', role: 'Engineer' }
// rejects when shouldFail:
Error('Failed to load user 1 (500)')
```

Users are keyed by stable numeric `id` so the card can use `key={userId}` to remount cleanly.

Log entry contract

```
{
  boundary: 'user-card',
  message: 'Failed to load user 1 (500)',
  stack: 'Error: Failed to load...\n at ...',
  componentStack: '\n at UserCard ...\n at ErrorBoundary ...',
  url: 'http://localhost:5173/',
  time: '2026-11-02T10:15:00.000Z'
}
```

6. Trace the data flow

Async fetch failure in the user card:

```
User checks "Fail request"
  -> App: setFailFetch(true) -> re-render
  -> UserCard effect: fetchUser(1, true)
  -> 600ms later promise rejects
  -> .catch: showBoundary(err) -> setError(err) in the hook
  -> UserCard re-renders -> hook throws err during render
  -> React walks up -> ErrorBoundary "user-card"
  -> getDerivedStateFromError -> { hasError: true, error }   (render phase)
  -> componentDidCatch -> logError(err, componentStack, 'user-card')   (commit phase)
  -> render(): <ErrorFallback error reset />  (role="alert" announced)
User unchecks "Fail request"
```

```
  -> resetKeys [1, true] -> [1, false] changed
  -> componentDidUpdate -> reset() -> UserCard remounts -> "Loading user…" -> user shown
```

7. Important technical challenges

Challenge 1: Boundaries don't catch async or event-handler errors

Problem: A `fetch` rejects inside `useEffect`, or a click handler throws. React only catches errors thrown *during render, lifecycles and constructors*. The promise rejection becomes an "unhandled rejection" and the UI stays stuck on "Loading…".

Solution: Save the error in state and throw it on the next render. Now it *is* a render error, so the nearest boundary catches it.

```
export function useErrorBoundary() {
  const [error, setError] = useState(null);
  const showBoundary = useCallback((err) => {
    setError(err instanceof Error ? err : new Error(String(err)));
  }, []);
  if (error) throw error;
  return { showBoundary };
}
```

For event handlers, decide per case: handle inline with `try/catch` (show a message), or call `showBoundary(err)` when the widget can't continue.

Challenge 2: Recovering (reset) without a page reload

Problem: After the fallback shows, clicking retry must remount the children with fresh state. Also, if the *input* that caused the error changes (another user is picked), the user should not have to click retry.

Solution: `reset()` clears boundary state, so children mount again (their old state is gone because the fallback had replaced them). For auto reset, compare `resetKeys` in `componentDidUpdate`. Skip the update that *just* caught the error, or the boundary would reset itself straight away.

```
componentDidUpdate(prevProps, prevState) {
  if (this.state.hasError && prevState.hasError &&
      haveKeysChanged(prevProps.resetKeys, this.props.resetKeys)) {
    this.reset();
  }
}

export function haveKeysChanged(prev = [], next = []) {
  return prev.length !== next.length ||
    prev.some((key, i) => !Object.is(key, next[i]));
}
```

Challenge 3: Where to put boundaries (granularity)

Problem: One boundary at the root means a broken counter blanks the whole dashboard. A boundary around every tiny component is noise and hides context.

Solution: One boundary **per independent feature** (widget, route, sidebar), plus a **root boundary** for anything that escapes. Controls that recover the widget sit *outside* its boundary.

```
<article className="card">
  <Controls />                      {/* survives the crash */}
  <ErrorBoundary name="user-card" resetKeys={[userId, failFetch]}>
    <UserCard key={userId} userId={userId} shouldFail={failFetch} />
  </ErrorBoundary>
</article>
```

Challenge 4: Logging in the right phase

Problem: `getDerivedStateFromError` runs in the render phase. It can run more than once (StrictMode, concurrent rendering), so side effects there can log twice or run for a render that never commits.

Solution: Keep `getDerivedStateFromError` pure (return state only). Log in `componentDidCatch`, which runs once in the commit phase and gets `info.componentStack`.

```
static getDerivedStateFromError(error) {
  return { hasError: true, error };          // pure
}
componentDidCatch(error, info) {
  logError(error, info.componentStack, this.props.name);   // side effect
  this.props.onError?.(error, info);
}
```

8. Implement in the correct order

#	Step	Working result
1	Vite app, `App` with three empty cards	Layout renders
2	`BuggyCounter` that throws at 3 (no boundary yet)	Show the whole app going blank: the problem
3	`ErrorBoundary` class with `getDerivedStateFromError` + plain fallback text	Only the counter card breaks
4	`ErrorFallback` with `role="alert"` and **Try again** -> `reset()`	Retry remounts the counter
5	`componentDidCatch` + `errorLogger`	Errors logged with component stack
6	Custom `fallback` prop (element or render function) + root boundary in `main.jsx`	App-level crash page
7	Mock `fetchUser` + `UserCard` with loading state	Card loads a user

#	Step	Working result
8	`useErrorBoundary` hook; `.catch(showBoundary)`	Fetch failure shows fallback
9	`resetKeys` + `haveKeysChanged`	Changing user / unchecking fail auto recovers
10	`SaveButton` : inline `try/catch` vs `showBoundary`	Event handler cases shown
11	Race guard (`ignore` flag), dev-only stack, focus styles	Edge cases covered

9. Cover all states and edge cases

State / case	Handling
Initial (no error)	Boundary returns `children` untouched
Loading	`UserCard` shows "Loading user..." with `aria-live="polite"`
Success	Widget content
Error (render)	Widget fallback with message + retry
Error (async)	`showBoundary` -> widget fallback
Error (event handler)	Inline message, or escalate with `showBoundary`
Error outside any widget	Root boundary full-page fallback
Retry but cause still there	Fallback appears again (deterministic); user changes input first
Input changes while in error	`resetKeys` auto reset
`throw null` / `throw 'text'`	`hasError` flag still shows fallback; message uses `String(error)`
Fallback throws	Goes up to the parent boundary (root)
Fetch resolves after unmount / user change	`ignore` flag drops the stale result
StrictMode double render	Logging only in `componentDidCatch` , so one log per caught error
Production	Stack trace hidden (`import.meta.env.DEV`)

10. Final structure

```
_40_error_boundries/
├── index.html
├── package.json
└── src/
    ├── main.jsx                  # root boundary + StrictMode
    ├── App.jsx                   # cards, one boundary per widget
    ├── App.css
    ├── errorBoundary/
    │   ├── ErrorBoundary.jsx     # class: catch, log, reset, resetKeys
    │   ├── ErrorFallback.jsx     # default accessible fallback
```

```
    │   └── useErrorBoundary.js     # async/event errors -> boundary
    ├── components/
    │   ├── BuggyCounter.jsx        # render error demo
    │   ├── UserCard.jsx            # async error demo
    │   └── SaveButton.jsx          # event handler error demo
    ├── services/
    │   ├── api.js                  # mock fetchUser
    │   └── errorLogger.js          # mock reporting
    └── utils/
        └── resetKeys.js            # haveKeysChanged
```

11. Full working code

src/errorBoundary/ErrorBoundary.jsx

```
import { Component } from 'react';
import ErrorFallback from './ErrorFallback';
import { logError } from '../services/errorLogger';
import { haveKeysChanged } from '../utils/resetKeys';

// hasError flag (not just `error`) so `throw null` / `throw undefined` still shows the fallback
const initialState = { hasError: false, error: null };

export default class ErrorBoundary extends Component {
  state = initialState;

  // render phase: switch to fallback UI (must be pure, no side effects)
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // commit phase: side effects like logging
  componentDidCatch(error, info) {
    logError(error, info.componentStack, this.props.name);
    this.props.onError?.(error, info);
  }

  componentDidUpdate(prevProps, prevState) {
    // prevState.hasError guard: don't reset in the same update that just caught the error
    if (
      this.state.hasError &&
      prevState.hasError &&
      haveKeysChanged(prevProps.resetKeys, this.props.resetKeys)
    ) {
      this.reset();
    }
```

```
  }

  reset = () => {
    this.props.onReset?.();
    this.setState(initialState);
  };

  render() {
    const { hasError, error } = this.state;
    const { fallback, children } = this.props;

    if (!hasError) return children;
    if (typeof fallback === 'function') return fallback({ error, reset: this.reset });
    if (fallback !== undefined) return fallback;
    return <ErrorFallback error={error} reset={this.reset} />;
  }
}
```

src/errorBoundary/ErrorFallback.jsx

```
export default function ErrorFallback({ error, reset, title = 'Something went wrong' }) {
  return (
    <div role="alert" className="fallback">
      <h3>{title}</h3>
      <p>{error?.message ?? String(error)}</p>
      {import.meta.env.DEV && error?.stack && (
        <details>
          <summary>Details</summary>
          <pre>{error.stack}</pre>
        </details>
      )}
      <button type="button" onClick={reset}>
        Try again
      </button>
    </div>
  );
}
```

src/errorBoundary/useErrorBoundary.js

```
import { useCallback, useState } from 'react';

// Boundaries only catch render errors. To route async / event-handler errors to the
// nearest boundary, store the error in state and re-throw it during the next render.
export function useErrorBoundary() {
  const [error, setError] = useState(null);
```

```
  const showBoundary = useCallback((err) => {
    setError(err instanceof Error ? err : new Error(String(err)));
  }, []);

  if (error) throw error;
  return { showBoundary };
}
```

src/utils/resetKeys.js

```
export function haveKeysChanged(prev = [], next = []) {
  return prev.length !== next.length || prev.some((key, i) => !Object.is(key, next[i]));
}
```

src/services/errorLogger.js

```
const logs = [];

// shortcut: in-memory + console only; swap for Sentry or navigator.sendBeacon('/api/logs') in prod
export function logError(error, componentStack, boundary = 'unknown') {
  const entry = {
    boundary,
    message: error?.message ?? String(error),
    stack: error?.stack,
    componentStack,
    url: window.location.href,
    time: new Date().toISOString(),
  };
  logs.push(entry);
  console.error(`[ErrorBoundary:${boundary}]`, entry);
  return entry;
}

export const getLogs = () => logs;
```

src/services/api.js

```
const USERS = {
  1: { id: 1, name: 'Ada Lovelace', role: 'Engineer' },
  2: { id: 2, name: 'Alan Turing', role: 'Researcher' },
  3: { id: 3, name: 'Grace Hopper', role: 'Admiral' },
};

// mock API: resolves after a delay, or rejects like a 500 when shouldFail is true
export function fetchUser(id, shouldFail = false) {
  return new Promise((resolve, reject) => {
```

```
    setTimeout(() => {
      if (shouldFail) reject(new Error(`Failed to load user ${id} (500)`));
      else resolve(USERS[id]);
    }, 600);
  });
}
```

src/components/BuggyCounter.jsx

```
import { useState } from 'react';

export default function BuggyCounter() {
  const [count, setCount] = useState(0);

  // render-phase error: the boundary catches this
  if (count === 3) throw new Error('Counter crashed at 3');

  return (
    <div>
      <p className="count">{count}</p>
      <button type="button" onClick={() => setCount((c) => c + 1)}>
        Increment (crashes at 3)
      </button>
    </div>
  );
}
```

src/components/UserCard.jsx

```
import { useEffect, useState } from 'react';
import { fetchUser } from '../services/api';
import { useErrorBoundary } from '../errorBoundary/useErrorBoundary';

// parent passes key={userId}, so a new user remounts the card with fresh loading state
export default function UserCard({ userId, shouldFail }) {
  const { showBoundary } = useErrorBoundary();
  const [user, setUser] = useState(null);

  useEffect(() => {
    let ignore = false; // drop results that arrive after unmount / user change
    fetchUser(userId, shouldFail)
      .then((data) => !ignore && setUser(data))
      .catch((err) => !ignore && showBoundary(err)); // async error -> nearest boundary
    return () => {
      ignore = true;
    };
  }, [userId, shouldFail, showBoundary]);
```

```
  if (!user) return <p aria-live="polite">Loading user…</p>;

  return (
    <div>
      <p className="user-name">{user.name}</p>
      <p className="muted">{user.role}</p>
    </div>
  );
}
```

src/components/SaveButton.jsx

```
import { useState } from 'react';
import { useErrorBoundary } from '../errorBoundary/useErrorBoundary';

function save() {
  throw new Error('Save failed: network down');
}

// Boundaries do NOT catch event-handler errors: handle them inline, or escalate on purpose
export default function SaveButton() {
  const { showBoundary } = useErrorBoundary();
  const [message, setMessage] = useState('');

  function handleInline() {
    try {
      save();
      setMessage('Saved');
    } catch (err) {
      setMessage(err.message);
    }
  }

  function handleEscalate() {
    try {
      save();
    } catch (err) {
      showBoundary(err);
    }
  }

  return (
    <div className="stack">
      <button type="button" onClick={handleInline}>
        Save (handle inline)
      </button>
```

```
      <button type="button" onClick={handleEscalate}>
        Save (escalate to boundary)
      </button>
      {message && (
        <p role="status" className="inline-error">
          {message}
        </p>
      )}
    </div>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import ErrorBoundary from './errorBoundary/ErrorBoundary';
import BuggyCounter from './components/BuggyCounter';
import UserCard from './components/UserCard';
import SaveButton from './components/SaveButton';
import './App.css';

export default function App() {
  const [userId, setUserId] = useState(1);
  const [failFetch, setFailFetch] = useState(false);
  const [crashApp, setCrashApp] = useState(false);

  // no widget boundary above this: the root boundary in main.jsx catches it
  if (crashApp) throw new Error('The whole app crashed');

  return (
    <main className="app">
      <header className="header">
        <h1>Error Boundaries</h1>
        <button type="button" className="danger" onClick={() => setCrashApp(true)}>
          Crash whole app
        </button>
      </header>

      <section className="grid">
        <article className="card">
          <h2>Render error</h2>
          <ErrorBoundary name="counter">
            <BuggyCounter />
          </ErrorBoundary>
        </article>

        <article className="card">
```

```
          <h2>Async error</h2>
          {/* controls live outside the boundary so they survive a crash */}
          <div className="controls">
            <label>
              User{' '}
              <select value={userId} onChange={(e) => setUserId(Number(e.target.value))}>
                <option value={1}>1</option>
                <option value={2}>2</option>
                <option value={3}>3</option>
              </select>
            </label>
            <label>
              <input
                type="checkbox"
                checked={failFetch}
                onChange={(e) => setFailFetch(e.target.checked)}
              />{' '}
              Fail request
            </label>
          </div>
          <ErrorBoundary name="user-card" resetKeys={[userId, failFetch]}>
            <UserCard key={userId} userId={userId} shouldFail={failFetch} />
          </ErrorBoundary>
        </article>

        <article className="card">
          <h2>Event handler error</h2>
          <ErrorBoundary name="save">
            <SaveButton />
          </ErrorBoundary>
        </article>
      </section>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import ErrorBoundary from './errorBoundary/ErrorBoundary';
import ErrorFallback from './errorBoundary/ErrorFallback';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    {/* last line of defence: reset remounts App with fresh state */}
```

```
    <ErrorBoundary
      name="root"
      fallback={({ error, reset }) => (
        <div className="app">
          <ErrorFallback error={error} reset={reset} title="The app crashed" />
        </div>
      )}
    >
      <App />
    </ErrorBoundary>
  </StrictMode>
);
```

src/App.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f5f6f8;
  color: #1f2328;
}

button {
  font: inherit;
  padding: 6px 12px;
  border: 1px solid #c7ccd1;
  border-radius: 6px;
  background: #fff;
  cursor: pointer;
}

button:focus-visible,
select:focus-visible,
input:focus-visible {
  outline: 2px solid #2563eb;
  outline-offset: 2px;
}

.app {
  max-width: 960px;
  margin: 0 auto;
  padding: 24px 16px;
}
```

```
.header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}

.danger {
  border-color: #dc2626;
  color: #dc2626;
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
  gap: 16px;
}

.card {
  background: #fff;
  border: 1px solid #e3e6ea;
  border-radius: 10px;
  padding: 16px;
}

.card h2 {
  margin-top: 0;
  font-size: 1.05rem;
}

.controls {
  display: flex;
  gap: 12px;
  margin-bottom: 12px;
}

.stack {
  display: grid;
  gap: 8px;
}

.count {
  font-size: 2rem;
  margin: 0 0 8px;
}
```

```
.user-name {
  font-weight: 600;
  margin: 0;
}

.muted {
  color: #6b7280;
  margin: 4px 0 0;
}

.inline-error {
  color: #b91c1c;
  margin: 0;
}

.fallback {
  border: 1px solid #fecaca;
  background: #fef2f2;
  border-radius: 8px;
  padding: 12px;
}

.fallback h3 {
  margin: 0 0 6px;
  color: #b91c1c;
}

.fallback pre {
  white-space: pre-wrap;
  font-size: 0.75rem;
  max-height: 160px;
  overflow: auto;
}
```

12. Wrap-up

Edge cases handled

- Render, async and event-handler errors, each routed the right way.
- Widget isolation plus a root safety net.
- Retry by button and auto reset by `resetKeys`.
- Non-Error throws (`null`, strings), stale async results, StrictMode double render.
- Dev-only stack trace; accessible fallback with `role="alert"`.

Trade-offs to mention

- **Class component is required:** there is no hook equivalent of `getDerivedStateFromError` / `componentDidCatch`. Everything else stays functional.

- **Own boundary vs `react-error-boundary`:** in production I would use the library (same API: `fallbackRender`, `resetKeys`, `useErrorBoundary`). Writing it here shows the internals.
- **Granularity:** more boundaries isolate better but cost more fallback design; I chose one per feature.
- **Retry with no cause change** just fails again; a smarter version counts retries and then suggests a reload.
- **Logger is in memory:** real apps batch and send with `navigator.sendBeacon` and remove duplicate errors.

With more time

- Tests with React Testing Library: a component that throws -> fallback appears -> retry -> children back; `resetKeys` change resets; `onError` called once.
- `withErrorBoundary(Component, props)` HOC for wrapping many components.
- Retry with backoff and a max retry count for flaky network widgets.
- Real reporting (Sentry), source maps, user/session context, sampling.
- Focus the fallback heading on mount for keyboard and screen reader users.
- Use React 19 `createRoot(el, { onCaughtError, onUncaughtError })` for global reporting.

13. Counter questions the interviewer may ask

Requirements

Q1. Which errors does an error boundary NOT catch?

Errors in event handlers, async code (`setTimeout`, promises, `fetch`), server-side rendering, and errors thrown in the boundary itself (those go to the parent boundary). It catches errors during render, in lifecycle methods and in constructors of its children.

Q2. Why not just wrap the whole app in one boundary?

Then any small widget crash replaces the entire page. Per-feature boundaries keep the rest of the page usable; the root boundary is only the last line of defence.

Q3. What happens if there is no boundary at all?

Since React 16, an uncaught render error unmounts the whole React tree, so the user sees a blank page. That is why at least a root boundary is needed.

State design

Q4. Why can't the boundary be a function component?

React exposes catching only through the class methods `getDerivedStateFromError` and `componentDidCatch`; there is no hook for them yet. So the boundary is the one class; everything else (fallback, hook, widgets) stays functional.

Q5. Difference between `getDerivedStateFromError` and `componentDidCatch`?

`getDerivedStateFromError` is static, runs in the render phase and only returns new state (show the fallback). `componentDidCatch` runs in the commit phase, gets `info.componentStack`, and is the place for side effects like logging.

Q6. Why keep a `hasError` flag instead of just checking `error`?

JavaScript can `throw null` or `throw undefined`. With only `error`, the boundary would think nothing went wrong and re-render the broken child forever.

Challenge: async errors

Q7. How does `useErrorBoundary` get an async error into the boundary?

It stores the error in state with `setError`. That causes a re-render, and the hook throws the error during render, which is something boundaries do catch.

Q8. Why not just use `window.onerror` / `unhandledrejection` for async errors?

Those are global: good for reporting, but they don't know *which* widget failed, so they can't show a local fallback. `showBoundary` targets the nearest boundary. I'd still add global listeners for logging.

Q9. Should every event-handler error go to the boundary?

No. If the user can continue (a failed save), show an inline message and keep the form. Escalate only when the widget can't work anymore.

Challenge: reset

Q10. How does Try again actually reset the child's state?

When the fallback renders, the children are unmounted, so their state is gone. `reset()` sets `hasError` back to false, the children mount fresh, and the counter starts at 0 again.

Q11. Why the `prevState.hasError` check in `componentDidUpdate`?

In the update where the error is first caught, `hasError` is already true. If the keys also changed in that same update, the boundary would reset at once and crash again in a loop. The check only resets errors that were already showing.

Q12. What if retry keeps failing?

A deterministic bug fails every time. I'd count retries in the boundary, and after N attempts show "Reload page" or "Contact support". For network errors, add exponential backoff.

Implementation

Q13. Why are the user select and checkbox outside the boundary?

If they were inside, they would disappear with the fallback and the user couldn't change the input that caused the error. Keeping them outside also makes `resetKeys` useful.

Q14. Why `key={userId}` on `UserCard`?

Changing the key remounts the card, so loading state resets for free with no `setUser(null)` inside the effect, and an old user is never shown for a new id.

Q15. Why the `ignore` flag in the fetch effect?

If the user switches from 1 to 2 quickly, the old request may resolve last and overwrite the new user, or call `showBoundary` on an unmounted card. The cleanup sets `ignore = true` so stale results are dropped. `AbortController` would also cancel the request itself.

Q16. How would you write a `withErrorBoundary` HOC?

```
export function withErrorBoundary(Comp, boundaryProps) {
  return function Wrapped(props) {
    return <ErrorBoundary {...boundaryProps}><Comp {...props} /></ErrorBoundary>;
  };
}
```

Performance and scale

Q17. Do boundaries slow down the app?

No. When there is no error, the boundary just returns `children`. The cost only appears when an error is caught.

Q18. What if 10,000 users hit the same error at once?

The logger must not flood the backend: remove duplicates by message + stack, sample (e.g. 10%), batch the reports, and send with `navigator.sendBeacon` so it survives page unload. Tools like Sentry group errors by fingerprint.

Q19. What if the API is slow?

The boundary isn't involved; the widget shows its loading state. For very slow calls add a timeout that rejects (then `showBoundary` shows the fallback) and a cancel with `AbortController`.

Trade-offs and "what if"

Q20. How do error boundaries work with Suspense?

Suspense handles the *loading* (a thrown promise), the error boundary handles the *failure* (a thrown error). Typical pattern: `<ErrorBoundary><Suspense fallback={<Spinner/>}><Widget/></Suspense></ErrorBoundary>`. With `use(promise)` in React 19, a rejected promise is thrown during render, so the boundary catches it with no hook.

Q21. What does React 19 add for error handling?

`createRoot(el, { onCaughtError, onUncaughtError, onRecoverableError })` gives app-wide hooks for reporting. Caught errors are no longer re-thrown, so each one is logged once instead of twice. The boundary pattern itself is unchanged.

Q22. How would you test this?

React Testing Library: render a child that throws, assert `getByRole('alert')`, click Try again, assert the child is back. Rerender with new `resetKeys` and assert reset. Mock `logError` and check it is called once. Silence

`console.error` in those tests.

Q23. Why not use `react-error-boundary` from the start?

In real code I would. It is small, tested, and has the same API. In the interview, writing it shows I understand the render/commit phases, reset, and why async errors need the re-throw trick.